

Roope Aaltonen

Pill Dispenser Project

Embedded Systems Programming Course Project

Metropolia University of Applied Sciences

Bachelor of Engineering

Information and Communications Technology

Embedded Systems Programming Project Report

15 May 2026

Abstract

Author:	Roope Aaltonen
Title:	Pill Dispenser Project
Number of Pages:	13 pages + 0 appendices
Date:	15 May 2026
Degree:	Bachelor of Engineering
Degree Programme:	Information and Communications Technology
Instructor(s):	Embedded Systems Programming course staff

This report describes the pill dispenser that I built with a Raspberry Pi Pico for the Embedded Systems Programming course. The device uses a stepper motor to move an eight-position dispenser wheel, an opto fork to find the reference position, a piezo sensor to detect a dropped pill, an AT24C256 EEPROM for saved state and a LoRa-E5 module for status messages. After calibration, the dispenser starts from SW0 and moves one pill position at a time with a 30 second interval. The most important software part was recovery after a reset during motor movement. Before a dispense movement starts, the program saves that a turn is in progress. If power is lost at that moment, the next boot reverses the wheel back to the opto reference, restores the last known position and finishes the interrupted dispense. I kept interrupt handlers short and put the actual decisions in the main loop.

Keywords:	Raspberry Pi Pico, stepper motor, EEPROM, LoRaWAN, GPIO interrupt, pill dispenser
-----------	--

Contents

1 Introduction	4
2 Hardware Connections	5
3 Program Structure	5
4 Code Examples	6
5 State Machine	7
6 Calibration	7
7 Dispensing and Piezo Detection	8
8 EEPROM Recovery	9
9 LoRaWAN Communication	9
10 Building, Running and Testing	10
11 Practical Test Results	11
12 Conclusion	12
13 References	13

1 Introduction

The aim of this project was to make a working embedded pill dispenser controller with the Raspberry Pi Pico and the second-year embedded systems board. I followed the course project handout for the required behaviour and the mechanical idea of the dispenser (Metropolia UAS, 2026a). The program controls the stepper-driven wheel, finds a reference position with the opto fork, checks pill drops with the piezo sensor, stores recovery data in EEPROM and sends important events through the LoRa-E5 module.

The project is implemented in C with the Raspberry Pi Pico SDK and its hardware libraries (Raspberry Pi Ltd., n.d.). The main source file is `project/Pill_dispenser/main.c` and the build target is `pill_dispenser`. In CLion, the run configuration flashes the Pico through OpenOCD and the debug probe. The monitor configuration is used for reading the debug output.

2 Hardware Connections

The hardware connections use the GPIO pins from the course board and project material (Metropolia UAS, 2026a; Metropolia UAS, 2026b). I kept UART0 for debug output and connected the LoRa module to UART1.

Function	GPIO pin	Purpose
SW0 button	GP9	User input for calibration and schedule start. Active low.
Status LED	GP20	Blinking wait indication, ready indication and error blinking.
Stepper coils	GP2, GP3, GP6, GP13	Four half-step outputs for the dispenser wheel motor.
LoRa-E5 UART	GP4 / GP5	UART1 at 9600 baud for AT commands.
EEPROM I2C	GP16 / GP17	I2C0 at 100 kHz for the AT24C256 EEPROM.
Piezo sensor	GP27	Falling-edge GPIO interrupt for pill drop detection.
Opto fork sensor	GP28	Active low reference signal for wheel calibration.

3 Program Structure

I divided the code into functions by hardware part and by application task. This makes the main loop easier to follow, because most low-level details are handled in their own functions.

- EEPROM functions read, validate and save the dispenser state with a checksum.
- GPIO initialization sets up the button, LED, opto fork, piezo interrupt and stepper outputs.
- Stepper functions output the half-step sequence and move the dispenser wheel.
- Calibration functions find the opto opening, measure one full revolution and center the wheel.
- LoRa functions send the required AT commands and short status messages.

- The main loop runs the application state machine and handles user interaction.

The only changing global runtime variable is `piezo_edges`. It has to be shared between the piezo interrupt handler and the main loop. The interrupt handler only increments this counter, so it stays short.

4 Code Examples

The following short code examples show the parts that are most important for the project: the piezo interrupt, saved motor-turn state, recovery after reset and LoRa reporting. The full code is in `project/Pill_dispenser/main.c`.

4.1 Minimal interrupt handler

```
static void gpio_irq_callback(uint gpio, uint32_t events) {
    if (gpio == PIEZO_PIN && (events & GPIO_IRQ_EDGE_FALL)) {
        piezo_edges++;
    }
}
```

The interrupt handler only records that the piezo input had a falling edge. It does not print text, use EEPROM or move the motor. The main loop later checks the counter and decides whether a pill was detected.

4.2 Saved turn state before motor movement

```
state->turn_in_progress = 1u;
(void)save_state(state);
lora_send_status(lora_joined, "turn_start", state->-pills_left);

bool detected = move_compartment(state, stepper_pins, current_step);

if (state->pills_left > 0u) {
    state->pills_left--;
}

state->turn_in_progress = 0u;
```

The EEPROM state is saved before the wheel starts moving. If power is lost during the movement, the next boot can see `turn_in_progress` and knows that the dispense had already started. The flag is cleared only after the compartment movement is finished.

4.3 Recovery after interrupted movement

```
if (!reverse_to_reference_opening(state, stepper_pins, current_step)) {
    state->mode = MODE_WAIT_CALIBRATION;
    return false;
}

uint32_t completed_dispenses = 7u - state->pills_left;
uint32_t restore_steps = completed_dispenses * steps_per_compartment(state);

if (restore_steps > 0u) {
    move_steps_forward(stepper_pins, current_step, restore_steps);
}
```

The recovery logic does not save every single motor step. After a reset during movement, the wheel first moves backwards to the known opto reference opening. Then it moves forward through the already completed positions and completes the interrupted dispense before continuing the normal schedule.

4.4 LoRaWAN status message

```
snprintf(message, sizeof(message), "%s left=%u", event, pills_left);
snprintf(command, sizeof(command), "AT+MSG=\"%s\"", message);
send_lora_command(command);
```

The LoRa message is a short status string with the event name and the remaining pill count. The string is sent to the LoRa-E5 module with AT+MSG. The AppKey is used when the module is configured, but it is not printed to the debug console.

5 State Machine

The dispenser uses three saved operating modes.

Mode	Meaning
Wait calibration	Initial state. The LED blinks and the system waits for SW0 before calibration.
Ready to start	Calibration is complete. The LED stays on and the system waits for the second SW0 press.
Running	The dispenser releases one compartment every 30 seconds until the wheel is empty.

The operating sequence I used is:

1. Load saved state from EEPROM after reset.
2. If a motor turn was interrupted, reverse to the calibration opening, restore position and complete the interrupted dispense immediately.

3. Wait for SW0 and calibrate the dispenser wheel.
4. Wait for SW0 again and start the 30 second schedule.
5. Move one compartment at each interval and check the piezo sensor result.
6. After seven positions, return to calibration mode for refilling.

6 Calibration

The opto fork sensor is active low. When the calibration opening reaches the sensor, the input changes from high to low. The program first detects this falling edge and then continues until the signal rises again. With these two edges, the program can measure the width of the opto opening.

After the rising edge, the wheel continues to the next falling edge. The measured opening width plus the distance to the next falling edge gives one full wheel revolution. The result is divided by eight because the dispenser wheel has eight pill positions.

After the revolution measurement, the motor moves half of the opto opening width forward. This centers the dispenser opening on the opto sensor instead of stopping at the first edge. Measurements that are clearly impossible are rejected, because then the opto fork is probably disconnected or wired incorrectly.

7 Dispensing and Piezo Detection

Before a compartment movement starts, the program clears the piezo edge counter. The piezo GPIO interrupt then counts falling edges while the motor turns and for a short time after the motor stops.

If at least one piezo falling edge is detected, the dispense is reported as successful. If no edge is detected, the status LED blinks five times. The remaining

pill position count is still reduced after the movement, because the wheel has physically moved to the next compartment even if the sensor did not detect a pill.

8 EEPROM Recovery

The AT24C256 EEPROM stores the current mode, pills left, measured steps per revolution, whether a motor turn is in progress and a checksum. The EEPROM is used through its two-wire serial interface, and the page-write behaviour follows the datasheet (Atmel Corporation, 2007). The program saves state before a turn starts and after the turn finishes, so it does not write to EEPROM on every motor step.

If reset happens while the wheel is moving, the next boot detects the interrupted turn. The recovery routine moves the wheel backwards until the calibration opening is found, centers on that opening, and then moves forward through the already emptied positions. After that, the interrupted dispense is completed immediately before the normal schedule continues. This avoids saving every motor step to EEPROM while still giving the dose that had already started.

9 LoRaWAN Communication

The LoRa-E5 module is configured with AT commands for OTAA mode, Class A operation, port 8 and DR5. The AT command format follows the LoRa-E5 command specification (Seeed Technology Co., Ltd., 2020). The AppKey comes from the course device list, but the program does not print the key to the debug console.

After joining the network, the application sends short status messages such as boot, calibration_start, calibrated, started, turn_start, dispensed, no_pill and empty. Each message also includes the number of pill positions left. Motor control still continues if LoRa is not available, because the dispenser should not stop only because the network is missing.

10 Building, Running and Testing

The project is built with the Pico SDK CMake target `pill_dispenser`:

```
cmake --build cmake-build-debug --target pill_dispenser -j 6
```

In CLion, the Run `pill_dispenser` + Monitor configuration flashes the target through the debug probe and opens the serial monitor. I used the monitor to check calibration, LoRa joining, schedule start and dispense events.

I verified the project with the following checks:

1. Built and flashed the `pill_dispenser` target through the debugger.
2. Opened the monitor and checked that the startup text appeared.
3. Pressed SW0 and checked that calibration started and reported the measured half steps.
4. Checked that calibration centered the wheel on the opto opening.
5. Pressed SW0 again and checked that the 30 second schedule started.
6. Checked that the stepper moved one compartment at each interval.
7. Tested both pill detected and no-pill cases with the piezo sensor.
8. Reset the device during motor movement and checked that the recovery logic returned the wheel to the reference position and continued the interrupted dispense.
9. Tested LoRaWAN status output in the school network with the provided receiver script.

11 Practical Test Results

I tested the complete setup with the assembled course board when LoRaWAN coverage was available at school. The LoRaWAN part could not be repeated later at home, because joining the network depends on the campus gateway and course network coverage. For that reason, the home test limitation is not treated as a software problem in this report.

In the school test, the serial monitor showed the LoRa-E5 setup sequence. The module answered the basic AT check with LoRa: +AT: OK, accepted OTAA configuration, joined the network and returned join lines including +JOIN: Network joined and +JOIN: Done. After joining, the program sent short AT+MSG status messages for boot, calibration, schedule start and dispensing events. This showed where the LoRa output can be seen and confirmed that the module was used through AT commands.

I tested calibration by pressing SW0. The wheel first searched for the opto fork reference opening and then measured one full revolution. In the working test, the measured value was about 4097 half steps per revolution, which gives 512 half steps per compartment for the eight-position wheel. The final version does not stop at the first opto edge. It moves to the middle of the measured opening, which made the dispenser hole line up better.

After calibration, I pressed SW0 again to start the dispensing schedule. The monitor showed the schedule start message and then one compartment movement every 30 seconds. During the test, a pill drop was not detected on every movement, so the no-pill path was also visible with output such as No pill detected, pills left: 6. The program still reduced the remaining position count after the mechanical movement, sent the event through LoRa and used the LED warning for the missing piezo detection.

EEPROM recovery was checked by restarting the device and confirming that the program restored the saved mode and pill count instead of always starting from an empty default state. The code saves turn_in_progress before the motor starts and

clears it only after the turn is complete. If power is lost during a movement, the next boot uses this flag to reverse back to the opto reference, return to the last completed position and finish the interrupted dispense immediately. This was important because the dispenser should give the dose that had already started, but it still avoids saving every motor step.

The main practical issue I noticed was mechanical alignment. A small visible offset can happen if the wheel, opto fork and dispenser hole are not physically mounted in exactly the same reference position. The software improves this by measuring the opto opening width and moving to its midpoint, but the final accuracy still depends on the physical assembly.

12 Conclusion

The finished project combines the main embedded systems topics from the course: GPIO input and output, interrupts, UART communication, I2C EEPROM access, stepper motor control, application flow and persistent recovery state. The code keeps the interrupt handler minimal and handles decisions in normal program flow, which made the system easier to debug.

The biggest remaining limitation is mechanical alignment. The software centers the wheel using the measured opto opening, but a small physical adjustment may still be needed if the dispenser hole and the sensor position are not perfectly aligned in the assembled mechanism.

13 References

Atmel Corporation. 2007. AT24C128/256: Two-wire Serial EEPROMs. Datasheet 0670T-SEEPR-3/07.

Metropolia University of Applied Sciences. 2026a. Pill dispenser. Embedded Systems Programming course project handout. Course material.

Metropolia University of Applied Sciences. 2026b. RPI second year board. Course hardware documentation. Course material.

Raspberry Pi Ltd. n.d. Raspberry Pi Pico C/C++ SDK documentation. Available at: <https://www.raspberrypi.com/documentation/pico-sdk/> (Accessed 15 May 2026).

Seeed Technology Co., Ltd. 2020. LoRa Wireless Module - Powered by STM32WLE5: AT Command Specification. Version V1.0.